MyCodeSchool

React — apeluri la backend

8 demo-uri · 40 de exerciții

Backend-urile: în `backend/` rulezi `docker compose up -d` (prima dată descarcă imaginile, durează câteva minute). Le oprești cu `docker compose down`; `docker compose down -v` șterge și datele, iar la pornire vin înapoi cei 5 utilizatori de start.

Aplicația: în folderul proiectului `npm install` (o dată), apoi `npm run dev` → `http://localhost:5173`.

Cum lucrezi: citești `TEORIE.pdf`, alegi în meniu demo-ul, îl citești în `src/demos/`, apoi rezolvi exercițiile lui în `src/exercitii/`. În cod scrii adresa cu prefixul backend-ului: `/users-api`, `/jobs-api`, `/shop-api`, `/school-api`. Ce trimite fiecare server vezi și în Swagger (tabelul din teorie).

Demo 1 — fetch la click

1a. Câți utilizatori — `Ex1aCount.tsx`

1. Un buton „Câți utilizatori?”.
2. La click: `GET /users-api/api/users`, apoi pui în stare `users.length`.
3. Sub buton apare propoziția doar după ce ai răspunsul.

Indiciu: stare `useState<number | null>(null)`; afișezi cu `{count !== null && ...}`.

Pe ecran

```
Sunt 5 utilizatori.
```

1b. Vârsta medie — `Ex1bAverageAge.tsx`

1. Buton „Vârsta medie” → `GET /users-api/api/users/average-age`.
2. Răspunsul e un obiect, nu o listă: `{ averageAge: 26.6 }`.

Indiciu: `type AverageAge = { averageAge: number }`.

Pe ecran

```
Vârsta medie: 26.6
```

1c. Top 3 cei mai în vârstă — `Ex1cTop3.tsx`

1. Buton „Top 3” → `GET /users-api/api/users/top3-by-age`.
2. Afișezi o listă numerotată `<ol>`: nume, prenume, vârstă.

Pe ecran

```
1. Bogdan Alexandrescu, 41 ani
2. Cristian Tudor, 30 ani
3. Maria Stere, 26 ani
```

1d. Emailurile — `Ex1dEmails.tsx`

1. Buton „Emailurile” → `GET /users-api/api/users`.
2. Listă `<ul>` doar cu emailul fiecăruia.

Indiciu: `key={user.id}` pe fiecare `<li>`.

Pe ecran

```
5 emailuri, unul pe rând
```

1e. Câți minori — `Ex1eMinors.tsx`

1. Buton „Câți minori?” → `GET /users-api/api/users/count?youngerThan=18`.
2. Răspunsul: `{ youngerThan: 18, count: 1 }`; afișezi `count`.

Pe ecran

```
Minori: 1
```

Demo 2 — useEffect la pornire

2a. Lista la pornire — `Ex2aStartList.tsx`

1. Fără buton: lista apare singură când deschizi exercițiul.
2. `GET /users-api/api/users` → `<ul>` cu numele complete.

Indiciu: funcția `load` scrisă întâi, apoi `useEffect(() => { load(); }, []);`

Pe ecran

```
5 nume, fără click
```

2b. Media la pornire — `Ex2bStartAverage.tsx`

1. La pornire: `GET .../average-age`.
2. Până vine răspunsul scrie „...”.

Indiciu: `{average === null ? "..." : average}`

Pe ecran

```
Vârsta medie: 26.6
```

2c. Adulții — `Ex2cAdults.tsx`

1. La pornire: `GET /users-api/api/users/adults`.
2. Un `<h2>` cu numărul lor și lista: nume, vârstă.

Pe ecran

```
Adulți: 4
```

2d. Utilizatorii de pe yahoo — `Ex2dYahoo.tsx`

1. La pornire: `GET /users-api/api/users/by-domain?domain=@yahoo.com`.
2. Listă cu emailurile primite.

Pe ecran

```
ana.stefanescu@yahoo.com
```

2e. Primul utilizator — `Ex2eFirstUser.tsx`

1. La pornire: `GET /users-api/api/users`; păstrezi doar primul.
2. Îl afișezi într-un `<div className="card">`: nume (h2), email, vârstă.
3. Dacă lista e goală: „Niciun utilizator”.

Indiciu: `useState<User | null>(null)`; `data[0]` doar dacă `data.length > 0`.

Pe ecran

```
Bogdan Alexandrescu
bogdan.alexandrescu@gmail.com
41 ani
```

Demo 3 — Se încarcă / eroare

3a. Se încarcă... — `Ex3aLoading.tsx`

1. Ca la 2a, plus o stare `loading`, pornită pe `true`.
2. Cât timp e `true`, componenta întoarce doar „Se încarcă...”.
3. După `setUsers`, o faci `false`.

Indiciu: `if (loading) { return <p>Se încarcă...</p>; }` înainte de `return`-ul cu lista.

Pe ecran

```
Se încarcă...  →  lista
```

3b. Serverul oprit — `Ex3bServerDown.tsx`

1. Ca la 2a, dar cererea stă într-un `try { ... } catch { ... }`.
2. În `catch` pui în stare mesajul și îl afișezi cu clasa `error`.
3. Probă: `docker compose stop users-api`, reîncarci pagina, apoi `docker compose start users-api`.

Indiciu: `catch` prinde doar când serverul nu răspunde deloc.

Pe ecran

```
Serverul nu răspunde. Pornește backend-ul.
```

3c. Email inexistent — `Ex3cNotFound.tsx`

1. Un input pentru email și un buton „Caută” → `GET /users-api/api/users/by-email?email=...`.
2. Dacă `response.ok`: numele userului.
3. Altfel: `status` + `message` din corpul erorii.

Indiciu: la eroare, corpul e `{ status, message, ... }`: tot cu `response.json()`.

Pe ecran

```
nimeni@mycode.ro      →  404: Email not found
ana.stefanescu@yahoo.com  →  Ana Stefanescu
```

3d. Reîncearcă — `Ex3dRetry.tsx`

1. Ca la 3b, iar lângă mesajul de eroare un buton „Reîncearcă” care cheamă din nou `load`.
2. La începutul lui `load` golești eroarea.

Pe ecran

```
oprit: mesaj + Reîncearcă  →  pornit + click: lista
```

3e. Statusul pe ecran — `Ex3eStatuses.tsx`

1. Trei butoane, fiecare cheamă aceeași funcție `call(url)` cu alt URL: `/users-api/api/users` · `.../by-email?email=nimeni@mycode.ro` · `.../older-than?age=abc`.
2. Afișezi doar `response.status`.

Indiciu: `onClick={() => call("...")}`

Pe ecran

```
Status: 200 · Status: 404 · Status: 400
```

Demo 4 — Parametri în URL

4a. Mai în vârstă decât — `Ex4aOlderThan.tsx`

1. Un input numeric (pornește de la 20) și buton „Caută”.
2. `GET /users-api/api/users/older-than?age=${age}` → „Găsiți: N” + lista.

Indiciu: URL-ul cu backtick-uri: `` `...?age=${age}` ``.

Pe ecran

```
25  →  Găsiți: 3
```

4b. După domeniu — `Ex4bDomain.tsx`

1. Input text (pornește de la `@gmail.com`) + buton „Caută” → `.../by-domain?domain=...`.
2. Lista emailurilor.

Pe ecran

```
@yahoo.com  →  ana.stefanescu@yahoo.com
```

4c. Între două vârste — `Ex4cBetweenAges.tsx`

1. Două inputuri numerice (18 și 30) + „Caută”.
2. `GET .../between-ages?minAge=...&maxAge=...`.

Indiciu: parametrii se leagă cu `&`.

Pe ecran

```
Maria Stere, 26 · Cristian Tudor, 30 · Ana Stefanescu, 19
```

4d. Emailul există? — `Ex4dEmailExists.tsx`

1. Input + „Verifică” → `GET .../exists?email=...`.
2. Răspunsul `{ email, exists }`: verde „există”, roșu „nu există”.

Indiciu: clasele `green` / `red` există în `index.css`.

Pe ecran

```
ana.stefanescu@yahoo.com: există
x@y.ro: nu există
```

4e. Pagini de câte 2 — `Ex4ePages.tsx`

1. La pornire: `GET .../search?page=0&size=2&sort=lastName,asc`.
2. Răspunsul: `{ content, page, totalPages, ... }`; afișezi `content`.
3. Butoanele „Înapoi” / „Înainte” cheamă `load(page - 1)` / `load(page + 1)`; dezactivate la capete.

Indiciu: `page` numără de la 0: pe ecran scrii `page + 1`.

Pe ecran

```
Pagina 1 din 3  →  Înainte  →  Pagina 2 din 3
```

Demo 5 — POST

5a. Creează un utilizator — `Ex5aCreate.tsx`

1. Formular: prenume, nume, email, parolă, vârstă (stări + inputuri controlate).
2. „Creează” → `POST /users-api/api/users` cu toate câmpurile în body.
3. La 201: „Creat: Prenume Nume (id)”; altfel „Status: N”.

Indiciu: `method`, `headers: { "Content-Type": "application/json" }`, `body: JSON.stringify({ firstName, ... })`. Parola are minim 8 caractere.

Pe ecran

```
Creat: Ion Pop (07ae82aa-...)
```

5b. Erorile 400 — `Ex5bValidationErrors.tsx`

1. Formular cu prenume, email, parolă (nume `"Test"` și vârstă 20 fixe în body).
2. La eroare: `message` + fiecare rând din `details` într-o listă roșie.

Indiciu: `type ApiError = { message: string; details: string[] }`.

Pe ecran

```
Validation failed
• email: Email is required
• firstName: First name is required
• password: Password must be at least 8 characters
```

5c. Email deja folosit — `Ex5cConflict.tsx`

1. Input email (pornește de la `ana.stefanescu@yahoo.com`) + „Creează” (restul câmpurilor fixe).
2. La 409: „Emailul e deja folosit: ” + `message`, cu roșu.

Indiciu: `if (response.status === 409)`

Pe ecran

```
Emailul e deja folosit: Email already used
```

5d. Apare în listă — `Ex5dAddToList.tsx`

1. La pornire încarci lista (ca la 2a) și afișezi „Utilizatori: N”.
2. Formular prenume, nume, email + „Adaugă” → POST.
3. La succes, userul primit de la server intră în listă fără să reîncarci.

Indiciu: `setUsers([...users, user])`, la fel ca la user-management-local.

Pe ecran

```
Utilizatori: 6  →  Adaugă  →  Utilizatori: 7
```

5e. Golește doar la succes — `Ex5eClearOnSuccess.tsx`

1. Formular prenume + email + „Salvează” → POST.
2. Succes: golești câmpurile și scrii „Salvat”.
3. Eroare: câmpurile rămân completate (să le poată corecta) și afișezi `message`.

Pe ecran

```
prenume gol → mesaj, emailul rămâne · corect → Salvat, câmpuri goale
```

Demo 6 — PATCH și DELETE

6a. Șterge — `Ex6aDelete.tsx`

1. Lista la pornire, cu un buton „Șterge” pe fiecare rând.
2. `DELETE /users-api/api/users/${id}`; dacă `response.ok`, îl scoți din listă.

Indiciu: `{ method: "DELETE" }`, fără body. Șterge doar useri creați de tine la ex. 5.

Pe ecran

```
8  →  Șterge  →  7
```

6b. –1 an — `Ex6bYounger.tsx`

1. Lista cu vârste și un buton „–1 an” pe fiecare rând.
2. `PATCH` cu body `{ age: user.age - 1 }`; userul întors de server îl pui în listă.
3. Butonul e dezactivat la 1 an.

Indiciu: `users.map((u) => (u.id === updated.id ? updated : u))`

Pe ecran

```
Ion Pop, 18 ani  →  Ion Pop, 17 ani
```

6c. Redenumește — `Ex6cRename.tsx`

1. Lista cu un buton „Alege” pe fiecare rând: pune userul în stare și prenumele lui într-un input.
2. „Salvează” → `PATCH` cu `{ firstName }` → lista actualizată.

Indiciu: PATCH trimite doar câmpul schimbat.

Pe ecran

```
Ion Pop  →  Ionel Pop
```

6d. Schimbă parola — `Ex6dPassword.tsx`

1. Un `<select>` cu userii (la pornire), un input de parolă și „Schimbă”.
2. `PUT /users-api/api/users/${userId}/password` cu body `{ newPassword }`.
3. Succes: „Parola a fost schimbată”; 400: rândurile din `details`.

Indiciu: `<option key={user.id} value={user.id}>`

Pe ecran

```
scurt  →  newPassword: Password must be at least 8 characters
parolanoua1  →  Parola a fost schimbată
```

6e. Șterge de două ori — `Ex6eDeleteTwice.tsx`

1. Lista cu „Șterge”, dar de data asta NU scoți userul din listă.
2. Afișezi `status` : `message` pentru fiecare ștergere.
3. Un buton „Reîncarcă lista”.

Indiciu: apasă de două ori pe același user: serverul știe că nu mai există.

Pe ecran

```
200: șters
404: User ID not found
```

Demo 7 — Job queue

7a. Trimite un job — `Ex7aSendJob.tsx`

1. Input pentru tip (pornește de la `email`) și „Trimite” → `POST /jobs-api/jobs` cu `{ type, payload: "Salut" }`.
2. Afișezi `jobId` din răspuns.

Pe ecran

```
Job: 59a4037c-9ede-...
```

7b. Verifică statusul — `Ex7bCheckStatus.tsx`

1. „Trimite” (ca la 7a) și „Verifică”, dezactivat până ai un `jobId`.
2. „Verifică” → `GET /jobs-api/jobs/${jobId}` ; `status` e un număr: 0 În coadă · 1 Rulează · 2 Gata · 3 Eșuat.

Indiciu: `const statusText: string[] = ["În coadă", "Rulează", "Gata", "Eșuat"]` → `statusText[job.status]`.

Pe ecran

```
imediat: Rulează · după 2 s: Gata
```

7c. Toate joburile — `Ex7cAllJobs.tsx`

1. La pornire și la „Reîncarcă”: `GET /jobs-api/jobs`.
2. „Joburi: N” + lista: tip și status în cuvinte.

Pe ecran

```
Joburi: 4
email: Gata
raport: Gata
```

7d. Verifică singur până e gata — `Ex7dAutoCheck.tsx`

1. Un singur buton: trimite jobul, apoi întreabă singur statusul la fiecare jumătate de secundă, până e 2 sau 3.
2. Fiecare status primit intră într-o listă numerotată.

Indiciu: `function wait(ms: number): Promise<void> { return new Promise((resolve) => setTimeout(resolve, ms)); }` apoi `while (status < 2) { await wait(500); ... }`. Copia listei: `setHistory([...seen])`.

Pe ecran

```
1. Rulează  2. Rulează  3. Rulează  4. Gata
```

7e. Trei joburi deodată — `Ex7eThreeJobs.tsx`

1. „Trimite 3 joburi”: un `for` peste `["email", "raport", "export"]`, câte un POST; păstrezi cele 3 id-uri.
2. „Câte sunt gata?”: un GET pe fiecare id și numeri câte au status 2.

Indiciu: `await` merge și în `for (const type of [...])`.

Pe ecran

```
Gata: 3 din 3
```

Demo 8 — Login + token

8a. Login la shop — `Ex8aShopLogin.tsx`

1. Inputuri email + parolă (pornesc cu `admin@shop.com` / `admin1234`) și „Login”.
2. `POST /shop-api/api/auth/login` cu `{ email, password }`; păstrezi `accessToken` în stare.
3. Afișezi primele 20 de caractere din token.

Indiciu: `token.slice(0, 20)`

Pe ecran

```
Logat ca admin@shop.com. Token: eyJhbGciOiJIUzI1NiJ9...
```

8b. Produsele cu token — `Ex8bProducts.tsx`

1. Butoanele „Login” (ca la 8a, cu datele fixe) și „Produse”.
2. „Produse” → `GET /shop-api/api/products` cu header-ul `Authorization`.
3. 200: „N produse” din `totalElements`; altfel „fără acces”.

Indiciu: `headers: { Authorization: \`Bearer ${token}\` }`

Pe ecran

```
înainte de login: 401: fără acces
după login: 200: 0 produse
```

8c. Parolă greșită — `Ex8cWrongPassword.tsx`

1. Input pentru parolă (emailul fix `admin@shop.com`) + „Login”.
2. Succes: „Logat” verde; altfel `status` + `message`, roșu.

Indiciu: și la 401 serverul trimite un JSON cu `message`.

Pe ecran

```
gresit  →  401: Invalid email or password.
admin1234  →  Logat
```

8d. Cursurile de la school — `Ex8dSchoolCourses.tsx`

1. Un buton „Cursurile”: întâi login la `POST /school-api/auth/login` (`admin@online-school.ro` / `password`), apoi, cu tokenul primit, `GET /school-api/courses`.
2. Lista: `courseName` (`departament`).

Indiciu: două `await fetch` unul după altul, în aceeași funcție; tokenul nu trebuie pus în stare.

Pe ecran

```
Algebra Fundamentals (Mathematics)
Java Programming Basics (Computer Science)
```

8e. Logout — `Ex8eLogout.tsx`

1. Un singur buton care e „Login” când nu ai token și „Logout” când ai.
2. „Logout” doar golește tokenul din stare.
3. „Produse” afișează statusul cererii cu tokenul curent.

Indiciu: `{token === "" ? <button ...>Login</button> : <button ...>Logout</button>}`

Pe ecran

```
Login → Produse: 200 · Logout → Produse: 401
```